Fase 1 — Análisis, modelado y requisitos

Sistema de Punto de Venta para Comercios Locales

Entrega académica

Octubre de 2026

Índice

1 Fase 1 — Sistema de Punto de Venta para Comercios Locales **2**
- 1.1 Problema y objetivo . . . 2
- 1.2 Alcance acordado . . . 2
- 1.3 Requerimientos funcionales y reglas verificables . . . 2
- 1.4 Requerimientos técnicos . . . 3
- 1.5 Decisiones y límites . . . 4
- 1.6 Para la defensa . . . 4

Diagrama de clases UML **5**

2 Diccionario de clases del dominio **6**
- 2.1 Relaciones del UML . . . 7
- 2.2 Para la defensa . . . 7

3 Pantallas previstas para el sistema **7**

1 Fase 1 — Sistema de Punto de Venta para Comercios Locales

1.1 Problema y objetivo

Un comercio que registra ventas e inventario manualmente puede vender existencias agotadas, perder el historial de precios y carecer de responsables identificables. El sistema centraliza productos, categorías, clientes, usuarios y ventas; descuenta inventario de forma atómica y permite consultar transacciones por fecha y responsable.

1.2 Alcance acordado

Aplicación de escritorio Java/Swing, MySQL y JDBC, organizada en MVC. Fase 2: lógica y vista de consola verificable; Fase 3: Swing. Sin proveedores, compras, devoluciones, facturación fiscal ni reportes avanzados. El precio incluye IVA: no se calcula ni desglosa el impuesto. Se registra el método de pago; no se procesa una tarjeta ni se almacenan sus datos.

Actores: administrador (todos los módulos) y cajero (consulta de productos, Caja, clientes desde Caja e historial propio). El administrador ve todas las ventas. La aplicación no usa una conexión MySQL distinta por persona; autentica usuarios de negocio con BCrypt.

1.3 Requerimientos funcionales y reglas verificables

ID	Requerimiento / regla	Criterio de aceptación
RF01	Autenticar usuarios activos	Clave incorrecta, usuario inexistente o inactivo rechaza el acceso con mensaje genérico.
RF02	Aplicar permisos en controladores	Un cajero no puede gestionar usuarios, categorías ni productos, aunque intente invocar la operación directamente.
RF03	CRUD de categorías	Nombre obligatorio y único; borrar una categoría con productos falla, incluso con productos inactivos.
RF04	CRUD y búsqueda de productos	Código único, nombre obligatorio, precio positivo con máximo 2 decimales, stock y mínimo enteros no negativos, categoría existente.
RF05	Desactivar productos vendidos	Sin ventas se permite DELETE; con ventas se conserva la fila y se marca activo=false.
RF06	Formar carrito con ArrayList	Agregar, agrupar el mismo producto, consultar y quitar; cantidades enteras positivas y sin exceder existencias.

ID	Requerimiento / regla	Criterio de aceptación
RF07	Confirmar venta atómicamente	Usuario activo, cliente existente y al menos un detalle; encabezado, detalles y stock se confirman juntos o se revierten juntos.
RF08	Evitar sobreventa concurrente	Dos cajas que compiten por la última unidad producen una sola venta exitosa.
RF09	Perecederos	Una fecha de vencimiento por producto. No vender después de esa fecha; de 0 a 3 días inclusive se descuenta 10 %, con HALF_UP a 2 decimales.
RF10	Conservar precio histórico	Cada detalle guarda el precio aplicado y subtotal; cambiar el catálogo no modifica importes históricos.
RF11	Consultar historial inclusivo por fechas	Administrador ve todas las ventas; cajero solo las propias; se muestran sus detalles.
RF12	Alertar stock bajo	Mostrar alerta cuando stock <= stockMinimo.
RF13	Gestionar clientes	Ambos roles pueden registrar y editar clientes desde Caja; solo administrador elimina clientes sin ventas. Consumidor Final (NIT CF) no se modifica ni elimina.
RF14	Gestionar usuarios	Administrador crea, consulta, edita y elimina; cuentas con ventas se desactivan. No puede borrar su cuenta ni quitarse su propio acceso administrativo.

1.4 Requerimientos técnicos

- Java camelCase y SQL snake_case; paquetes `model`, `view`, `controller`, `dao`, `util` bajo `com.progra.pos`.
- Modelos encapsulados con atributos privados; constructores validan invariantes; modificación del catálogo mediante nuevos objetos y UPDATE explícito.
- Herencia y polimorfismo en usuarios y productos. ArrayList en carrito, detalles y resultados DAO.
- Todo SQL de la aplicación usa PreparedStatement; entradas del usuario mediante parámetros. Fragmentos SQL condicionales solo provienen de constantes internas.
- JDBC con try-with-resources. SQLException se traduce a AccesoDatosException; la vista muestra un mensaje sin exponer consultas ni credenciales. Rollback conserva la excepción original.
- Contraseñas BCrypt, factor 12 y salt por hash. Mínimo 8 caracteres y máximo 72 bytes UTF-8; no se truncan silenciosamente.
- BigDecimal ↔ DECIMAL(12,2); LocalDate ↔ DATE; LocalDateTime ↔ DATETIME. Reloj local del comercio: no mezclar zonas horarias entre cajas.
- No se guardan contraseñas de MySQL en Git. Variables POS_DB_URL, POS_DB_USER, POS_DB_PASSWORD.

1.5 Decisiones y límites

`Venta.usuario` referencia Usuario, no exclusivamente Cajero: el administrador también vende. `Venta` en construcción puede estar vacía; el DAO no guarda ventas vacías. No hay edición ni eliminación de ventas confirmadas. La composición es conceptual y no exige borrado SQL en cascada: aquí se protege la auditoría con RESTRICT.

El carrito muestra un total estimado; al confirmar acepta precios vigentes, calculados nuevamente dentro de la transacción. Si cambia el precio mientras se prepara el carrito, el importe definitivo puede cambiar. El historial conserva precio y cantidad, pero muestra los nombres actuales de producto y cliente: no es un comprobante fiscal inmutable.

Se admite una sola fecha de vencimiento por código. Lotes con fechas diferentes requieren una ampliación fuera del alcance. El NIT se valida como texto obligatorio y único, sin validación tributaria oficial. No hay recuperación de contraseña, MFA ni límite de intentos: alcance académico local.

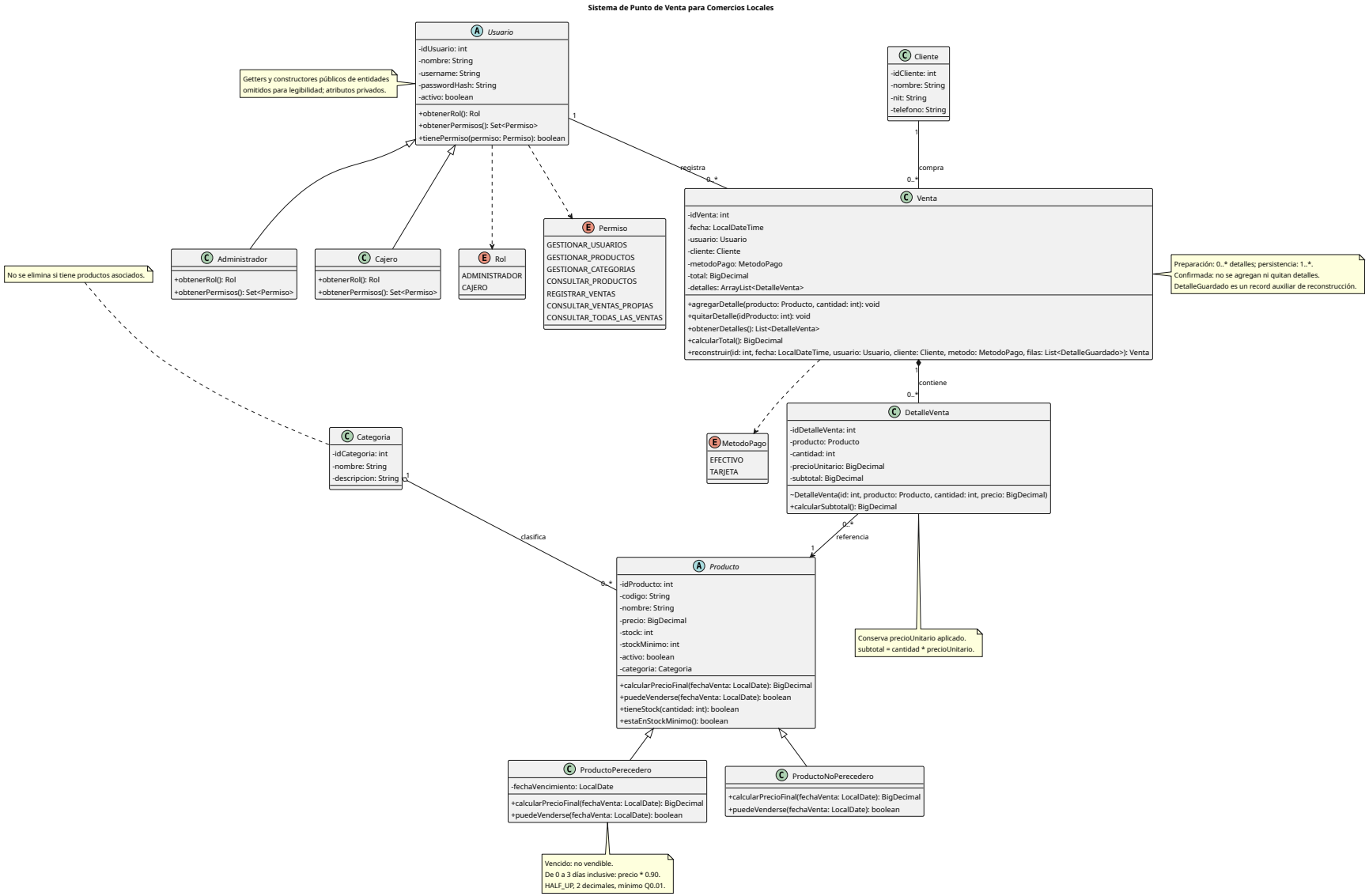
1.6 Para la defensa

Explicación: una regla debe indicar qué puede ocurrir y cómo demostrar que se cumple. Las validaciones del modelo previenen entradas inválidas; las restricciones SQL protegen los datos; la transacción protege el conjunto de cambios.

1. **¿Por qué validar en Java y SQL?** Java ofrece mensajes tempranos; SQL impide guardar datos inválidos incluso si otro cliente accede a la BD.
2. **¿Por qué Usuario en Venta?** Ambos roles pueden registrar ventas, y el historial necesita identificar al responsable real.
3. **¿El stock del carrito garantiza la venta?** No. Otra caja puede comprar antes; se verifica nuevamente con bloqueo dentro de la transacción.

Cambio en vivo: mostrar alerta tres unidades antes del mínimo. 1) Abrir Producto.java. 2) Cambiar la comparación en estaEnStockMinimo a `(long) stock <= (long) stockMinimo + 3`. 3) Actualizar RF12. 4) Probar mínimo=5 con stock 8 y 9; solo 8 debe alertar. Convertir a long evita desbordamiento al sumar.

Diagrama de clases UML



2 Diccionario de clases del dominio

Ruta de las clases: `src/main/java/com/progra/pos/model/`.

Clase	Responsabilidad y atributos principales	Métodos clave
Usuario (abstracta)	Identidad y autenticación: idUsuario:int, nombre/username/passwordHash:String, activo:boolean.	obtenerRol, obtenerPermisos, tienePermiso.
Administrador	Especialización de Usuario con todos los permisos.	Sobrescribe obtenerRol y obtenerPermisos.
Cajero	Especialización de Usuario limitada a consulta, Caja e historial propio.	Sobrescribe obtenerRol y obtenerPermisos.
Categoria	Agrupación: idCategoria:int, nombre/descripcion:String.	Constructor validado, getters.
Producto (abstracta)	Catálogo: idProducto:int; codigo/nombre:String; precio:BigDecimal; stock/stockMinimo:int; activo:boolean; categoria:Categoria.	calcularPrecioFinal, puedeVenderse, tieneStock, estaEnStockMinimo.
ProductoPerecedero	Agrega fechaVencimiento:LocalDate; valida vigencia y descuento.	Sobrescribe calcularPrecioFinal y puedeVenderse.
ProductoNoPerecedero	Mantiene precio base y exige estar activo.	Sobrescribe calcularPrecioFinal y puedeVenderse.
Cliente	Comprador: idCliente:int, nombre/nit/telefono:String.	Constructor validado, getters.
Venta	Encabezado: idVenta:int, fecha:LocalDateTime, usuario:Usuario, cliente:Cliente, metodoPago:MetodoPago, total:BigDecimal, detalles:ArrayList.	agregarDetalle, quitarDetalle, calcularTotal, obtenerDetalles, reconstruir.
DetalleVenta	Línea: idDetalleVenta:int, producto:Producto, cantidad:int, precioUnitario/subtotal:BigDecimal.	calcularSubtotal; constructor de paquete coordinado por Venta.
Rol (enum)	ADMINISTRADOR, CAJERO.	valueOf, name heredados de Enum.
Permiso (enum)	Capacidades explícitas de cada rol.	Valores usados por Sesion.exigir.
MetodoPago (enum)	EFECTIVO, TARJETA.	valueOf, name.

`Venta.DetalleGuardado` es un record auxiliar de reconstrucción JDBC; no es otra tabla. Transporta id:int, producto:Producto, cantidad:int y precio:BigDecimal para restaurar el precio histórico. Venta confirmada no permite agregar ni quitar detalles. Las listas devueltas son copias no modificables.

2.1 Relaciones del UML

- Herencia: Administrador/Cajero → Usuario y Perecedero/NoPerecedero → Producto (`extends`).
- Agregación: Categoria 1 ◇— 0..* Producto; cada producto tiene exactamente una categoría y puede reasignarse.
- Composición: Venta 1 ◆— 0..* DetalleVenta durante preparación; al persistir exige 1..*. Cada detalle tiene una única venta propietaria.
- Asociación: Usuario 1 — 0..* Venta y Cliente 1 — 0..* Venta.
- Asociación navegable: DetalleVenta 0..* → 1 Producto.

2.2 Para la defensa

Explicación: encapsular no exige crear setters para todo. Los objetos del catálogo son inmutables; editar significa construir una versión validada y enviarla al DAO. Venta sí cambia durante la preparación y controla sus detalles.

1. **¿Por qué Producto es abstracta?** No hay un producto genérico vendible sin decidir su comportamiento respecto del vencimiento.
2. **¿Dónde se usa ArrayList?** En detalles de Venta, líneas de VentaController y listas devueltas por los DAO.
3. **¿Por qué obtenerDetalles no devuelve la lista interna?** Para impedir que un llamador agregue líneas sin validar cantidad, precio y stock.

Cambio en vivo: descuento de 15 % hasta cinco días antes. 1) Abrir ProductoPerecedero.java. 2) Cambiar `dias <= 3` por `dias <= 5`. 3) Cambiar `0.90` por `0.85`. 4) Actualizar ModeloTest con días 0, 5, 6 y -1; Q100 debe resultar Q85, Q85, Q100 y rechazo. 5) Actualizar RF09 y la nota UML.

3 Pantallas previstas para el sistema

La Fase 1 deja definido el alcance visual esperado. La implementación de escritorio se incorporó en la Fase 3 con Java Swing; esta lista documenta qué verá cada rol y cómo se relacionan las pantallas con los requisitos.

Pantalla	Usuario	Funciones previstas	Requisitos relacionados
Inicio de sesión	Administrador y cajero	Identificar usuario activo y abrir el menú según permisos.	RF01–RF02
Menú principal	Administrador y cajero	Navegar únicamente a módulos autorizados.	RF02
Productos	Administrador; consulta para cajero	Buscar y mantener productos, categoría, precio, existencia, mínimo y vencimiento; alertar existencia baja.	RF04–RF05, RF09, RF12
Categorías	Administrador	Crear, consultar, editar y borrar categorías sin productos asociados.	RF03

Pantalla	Usuario	Funciones previstas	Requisitos relacionados
Caja / nueva venta	Administrador y cajero	Elegir cliente y método de pago; agregar, agrupar y quitar productos; revisar total y confirmar venta.	RF06–RF10, RF13
Clientes desde Caja	Administrador y cajero	Registrar y editar clientes durante el cobro; aplicar restricciones para Consumidor Final y borrado.	RF13
Historial de ventas	Administrador y cajero	Filtrar por fechas; administrador ve todas, cajero solo propias; consultar detalles.	RF11
Usuarios	Administrador	Crear, consultar, editar y eliminar o desactivar cuentas según historial.	RF14

El producto acordado es una aplicación de escritorio Swing, no una página web. La consola de Fase 2 permite probar la lógica; la interfaz gráfica es la fase posterior.